EE147 Exam 2 — Flashcard Drill

Active recall set: Lectures 8–14

Kaushik Vada

How to drill: cover each answer with a card. Speak the answer out loud before checking. Loop the deck twice. Skip cards you nail; star ones you miss.

Unified Memory (Lec 8)

Q. What CUDA call replaces both `malloc` and `cudaMalloc` in a UM-style program?
A. `cudaMallocManaged(&ptr, size)`.

Q. Pre-Pascal: what triggers the data to come back to the CPU after a kernel?
A. `cudaDeviceSynchronize()` — until you call it, the CPU touching the managed pointer would segfault.

Q. Pascal+: what happens when the GPU touches a page that lives in CPU memory?
A. Page fault → driver migrates that page to GPU → the access is replayed. Migration is page-by-page on demand.

Q. Why is page-by-page demand paging slow for bulk data?
A. Each page fault has fixed overhead. A single bulk `cudaMemPrefetchAsync` or `cudaMemcpy` is much more efficient.

Q. Three `cudaMemAdvise` hint values?
A. `cudaMemAdviseSetReadMostly`, `cudaMemAdviseSetPreferredLocation`, `cudaMemAdviseSetAccessedBy`.

Q. Do `cudaMemAdvise` hints move data?
A. No. They guide future migration/mapping decisions. Only `cudaMemPrefetchAsync` actually moves data on demand.

Q. What does `cudaMemAdviseSetReadMostly` do?
A. UM keeps a read-only copy local to each processor that touches it. A write collapses all copies into one and subsequent reads re-fault and re-duplicate.

Q. Pascal+: can CPU read managed memory while a kernel is running on it?
A. Yes (page fault, then resolve). On pre-Pascal this would crash — segfault.

Q. What's special about `atomicAdd_system`?
A. System-wide atomic — coherent across CPU and all GPUs. Direct over NVLink, software-assisted over PCIe. Pascal+ only.

Q. Can you oversubscribe GPU memory with UM on Kepler?
A. No. Oversubscription requires Pascal+. On Kepler, allocating more than GPU memory fails.

Matrix Multiply / Tiling (Lec 9)

Q. In a basic matmul kernel, how many global memory loads does one thread do?
A. $2 \cdot \text{Width}$ — one row of M, one column of N, both per inner-loop step.

Q. What's the compute-to-memory ratio in a tiled SGEMM with TILE_WIDTH = 16?
A. ∼16 FLOPs per global memory load (per phase: 512 loads, 8192 mul/adds for a 256-thread block).

Q. TILE_WIDTH = 32 — shared memory per block?
A. $2 \cdot 32 \cdot 32 \cdot 4 =$ **8 KB** (two tiles, each 32×32 floats).

Q. On a 16 KB shared memory SM, how many tile = 32 blocks can be co-resident?
A. 2 blocks (each needs 8 KB).

Q. Why two `__syncthreads()` per phase in tiled SGEMM?
A. First: ensure tile is fully loaded before consumption. Second: ensure tile is fully consumed before loading the next one (otherwise a fast thread could overwrite data still in use).

Q. In the tiled kernel, what does each thread load per phase?
A. One element of the M tile and one element of the N tile — total of 2 loads.

Q. Boundary check for loading the M tile in a non-multiple-size matrix?
A. `(Row < Height) && (p*TILE_WIDTH + tx < Width)` — if true, load; else write 0.

Q. Why write 0 when out of bounds instead of skipping the load?
A. So the multiply-add becomes a no-op without breaking other threads' computations or causing branch divergence after sync.

Q. Are the three boundary tests (load M, load N, write P) the same?
A. No — they're all different. A thread that doesn't write a valid P can still participate in loading.

Memory Coalescing (Lec 10)

Q. Rule of thumb for an access to be coalesced?
A. The index in `A[expr]` should be `(something not involving threadIdx.x) + threadIdx.x`.

Q. In a row-major 2D array `M[i][j]`, which dimension should track `threadIdx.x` for coalesced reads?
A. The column index `j`. Consecutive threads (varying tx) then hit consecutive addresses in memory.

Q. In basic matmul, are loads of `N[k*Width + Col]` coalesced?
A. Yes — `Col` = `bx*TILE_WIDTH + tx`, so adjacent tx → adjacent address.

Q. What is a DRAM "burst section"?
A. A contiguous range of addresses transferred together. Threads in a warp accessing one burst section = one DRAM request. Spreading across sections = multiple requests.

Q. Why does the tiled SGEMM kernel still benefit from coalescing even though it uses shared memory?
A. The *loads from global into shared* are still coalesced; only the inner consumption loop reads from shared (where coalescing doesn't apply).

Q. P100 memory? V100? A100? H100?
A. P100: HBM2 732 GB/s. V100: HBM2 900 GB/s. A100: HBM2 1555 GB/s. H100: HBM3 3 TB/s.

Q. What does MIG let you do?
A. Slice a GPU into multiple independent instances, each with its own SMs, memory partition, and L2 slice — multi-tenant isolation.

Histogram / Atomics / Privatization (Lec 11)

Q. Sectioned vs interleaved partitioning of the input — which is coalesced?
A. Interleaved. Threads start at consecutive positions and stride together. Sectioned has thread i on a distant chunk → not coalesced.

Q. Why is read-modify-write a data race?
A. Two threads can both read the old value, both increment to old+1, both write back. Only one of the increments is visible.

Q. What does `atomicAdd` return?
A. The *old* value (before the add).

Q. Latency tiers for atomic ops: DRAM vs L2 vs shared?
A. DRAM: ∼1000+ cycles. L2: ∼1/10 of DRAM. Shared: very short (no off-chip traffic).

Q. Why does heavy atomic contention on one address devastate throughput?
A. All ops serialize on that location. Throughput becomes 1/(read-latency + write-latency), not bandwidth-limited.

Q. What's the core privatization idea for histogram?
A. Each block builds a private histogram in shared memory (fast atomics, low contention). At the end, atomically merge each block's private bins into the global histogram.

Q. When can you NOT use privatization?
A. When the reduction op isn't associative + commutative, or when the private structure can't fit in shared memory.

Q. What's the speedup typically reported for privatization?
A. Often >10×.

Q. In the privatized histogram kernel, what syncs are needed?
A. One `__syncthreads()` after initializing private bins, one after building the private histogram, before merging into the global one.

Multi-GPU (Lec 12)

Q. How do you target a specific GPU from a single CPU thread?
A. `cudaSetDevice(i)` — sets the current device. Async calls keep running on previous devices.

Q. Function to check if peer access is possible between two devices?
A. `cudaDeviceCanAccessPeer(&result, dev_X, dev_Y)`.

Q. Function to enable P2P access?
A. `cudaDeviceEnablePeerAccess(peer_device, 0)`.

Q. What's `cudaMemcpyPeerAsync`?
A. Copies bytes from one device to another. With P2P enabled: shortest PCIe path, no host staging. Without P2P: driver stages through host memory.

Q. Why two stages in the Left-Right exchange?
A. So no PCIe link carries traffic in the same direction twice at the same time — Stage 1 = all send right, Stage 2 = all send left. The 3 transfers in each stage run concurrently.

Q. Local vs remote D2H bandwidth on a dual-IOH system?
A. Local ∼6.3 GB/s, remote ∼4.3 GB/s. Remote pays one QPI hop.

Q. Why can't P2P cross IOHs on dual-IOH systems?
A. QPI between IOHs isn't compatible with PCIe P2P protocol. Copies still work but stage through host memory.

Q. How do you pin a CPU thread to the socket closest to a GPU?
A. `numactl`, `GOMP_CPU_AFFINITY`, `KMP_AFFINITY`, or similar.

GPU Sharing / MPS (Lec 13, up to slide 25)

Q. What problem does MPS solve?
A. Without MPS, kernels from different processes serialize at context-switch boundaries on a shared GPU. With MPS, they share one context and kernels can overlap, improving utilization.

Q. What's Hyper-Q?
A. A hardware feature (Kepler+) providing 32 work queues, one per stream. Allows true 32-way kernel concurrency without inter-stream false dependencies.

Q. MPS requirements?
A. Linux. UVA. Tesla GPU with CC $\geq$ 3.5. CUDA toolkit 5.5+. Exclusive-mode applies to the MPS server, not clients.

Q. Do you need to modify your application to use MPS?
A. No. Run the MPS daemon `nvidia-cuda-mps-control -d`; processes started after that route through it automatically.

Q. When does MPS introduce false dependencies / serialization?
A. When the number of streams per MPS client exceeds the channels available (more than 2 per client). Excess streams get squeezed into the same work queue and serialize.

Q. Does MPS spread work across multiple GPUs?

A. No — the application must still handle GPU affinity itself (e.g., set `CUDA_VISIBLE_DEVICES` per rank).

Q. Quick setup command sequence on a single GPU?
A. `export CUDA_VISIBLE_DEVICES=0` ; `nvidia-smi -i 0 -c EXCLUSIVE_PROCESS` ; `nvidia-cuda-mps-control -d`.

Dynamic Parallelism (Lec 14)

Q. What does Dynamic Parallelism enable?
A. A GPU kernel can launch its own child kernels — dynamically, simultaneously, and independently from the host.

Q. Minimum compute capability?
A. 3.5 (Kepler GK110 and later).

Q. If 256 threads in a block all execute the launch line, how many child grids are created?
A. 256. Launches are per-thread. Guard with `if (threadIdx.x == 0)` to launch once per block.

Q. Does `cudaDeviceSynchronize()` on the device synchronize only the calling thread?
A. No — it waits for all child grids launched by any thread in the block.

Q. Does `cudaDeviceSynchronize()` act as `__syncthreads()`?
A. No. If you need a barrier after the children finish, you must also call `__syncthreads()`.

Q. Are device-side launches synchronous?
A. No — all device-side launches and copies are async.

Q. Three problem types Dynamic Parallelism is useful for?
A. Library calls from inside kernels, data-dependent parallelism (irregular loop bounds), recursive parallel algorithms. Also batched nested parallelism and adaptive grid refinement.

Q. Can you pass a CUDA stream to a child kernel from inside a kernel?
A. No — CUDA primitives (streams, events) are per-block; can't be passed to children.

Q. Where must constants and texture/surface bindings be set?
A. From the host. ECC errors also report at host.

Q. Why does DP help an irregular workload like `for j = 1 to x[i]`?
A. The static alternatives are wasteful — either oversubscribe with the worst-case grid and waste cycles on empty cells, or serialize per row. DP lets each row spawn exactly the right child grid.

Mixed / cross-cutting

Q. Warp size on NVIDIA GPUs?
A. 32 threads.

Q. Difference between `cudaMemcpy` (sync) and `cudaMemcpyAsync`?
A. Sync blocks until the copy completes. Async returns immediately and proceeds on the given stream. Async H2D requires pinned host memory.

Q. Why does pinned memory help bandwidth on H2D copies?
A. Pinned (page-locked) memory enables the DMA engine to copy directly without CPU-driver staging through pinned buffers.

Q. What's the maximum threads per block on modern GPUs?
A. 1024.

Q. If a 2D block is 32×32, how many warps per block?
A. 32 (1024 threads / 32 threads/warp).

Q. For an image of $H \times W$ with block 16×16, what's the grid?
A. `gridDim.x = ceil(W/16)`, `gridDim.y = ceil(H/16)`.

Q. How does shared memory size affect occupancy?

A. It limits how many blocks can be co-resident on one SM. More shared per block → fewer blocks → less latency hiding.

End of deck. Loop twice; star the misses; rotate the stars.